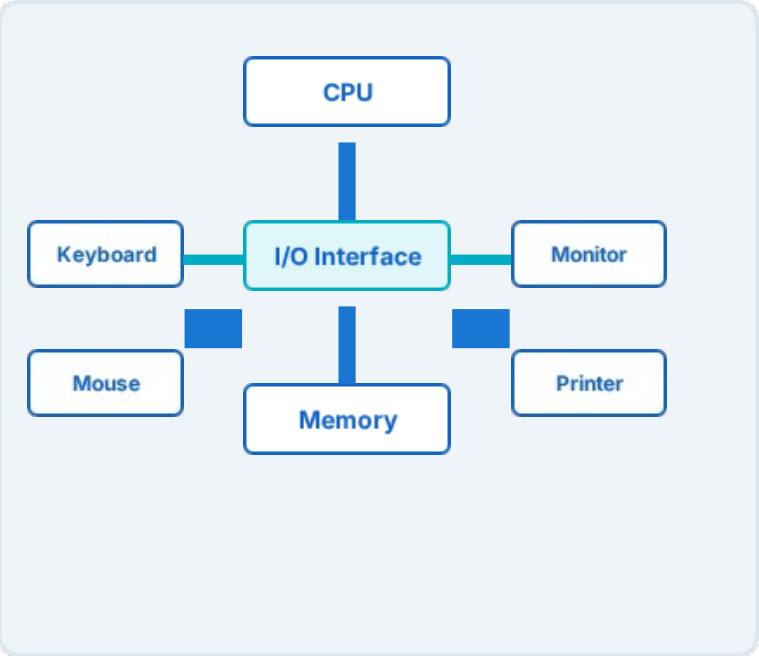
UNIT 4 PRESENTATION

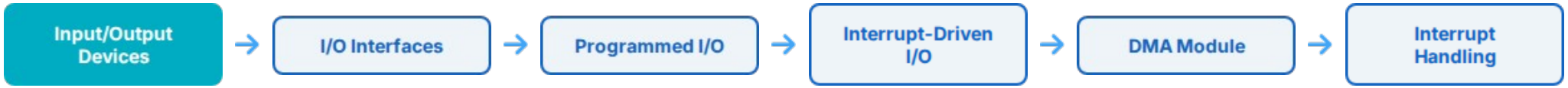
INPUT/OUTPUT ORGANIZATION

Subject: Computer Organization
Department: Newton School of AI & Computing
Presented By: Deepanshu Saini
Academic Session: 2026



UNIT 4 SYLLABUS ROADMAP

An organized logical hierarchy of all the core chapters and computational topics in Unit 4.



Hardware Components

- Common peripherals (Keyboard, Mouse, Monitor, Printer).
- Need and logical structure of the **I/O Interface Module**.
- Status, Control, and Data Register coordination.

Data Transfer & Control

- CPU-driven communication methods (Polling / Programmed).
- Asynchronous Interrupt requests and Hardware routines.
- High-speed block bypass logic with **Direct Memory Access (DMA)**.

INDUSTRY APPLICATIONS OF I/O SYSTEMS

Why Efficient I/O is Critical

Modern computation is heavily bottlenecked by input-output pathways. A fast processor sitting idle waiting for storage data represents major industrial losses.

Real-World Implementations

- **Automotive Electronics:** Millisecond response to sensor inputs via prioritized interrupt systems.
- **Data Centers:** Processing millions of user requests through DMA-driven storage.
- **Embedded Systems:** Optimized low-power interrupt handlers in smart IoT devices.

COURSE OBJECTIVES (NEWTON SCHOOL)

APPROVED COURSE OUTCOMES

ENGINEERING PROGRAM OUTCOMES

COURSE PROGRAM MAPPING MATRIX

PROGRAM SPECIFIC OUTCOMES

CO-PSO MAPPING MATRIX

PROGRAM EDUCATIONAL OBJECTIVES

CLASS PERFORMANCE ANALYTICS

UNIVERSITY EXAMINATION BLUEPRINT

PREREQUISITE FOUNDATIONS

INTRODUCTION TO I/O ORGANIZATION

Core Philosophy:

"Input devices provide data to the computer, while output devices communicate processed information to the user or another system."

The Coordination Layer

Without an I/O organization, the central processing unit would be locked to physical electromechanical hardware, reducing its speed down to the physical speed of key presses or printing cylinders.

TOPIC VIDEO LEARNING RESOURCE

NPTEL University Lecture Series

Topic: Introduction to System I/O Modules & Speed Buffering Schemes

Speaker: Neso Academy / Recognized University Academic Faculty

Target Objective: Gain three-dimensional visualization of bus timing clocks and data ready lines during peripheral reads.

Verified Online Resource Reference

Use the following authentic academic URL to watch live hardware visual demonstrations:

https://www.nesoacademy.org/cs/computer-organization-and-architecture

Please scan the institutional library registry system to log your video access points.

COMPLETE UNIT CONTENTS MAP

1. Input/Output Peripherals

- Keyboard & Mouse systems.
- Monitor structures (Pixels, LED).
- Printers (Impact / Non-Impact).

2. Interface Architecture

- The Need for Interface Chips.
- Status Register bits.
- Control Commands.

3. Communication Schemes

- Programmed (Polling) Loop.
- Interrupt Vector Table.
- DMA cycle transfers.

LEARNING OBJECTIVES FOR UNIT 4

Core Course Outcomes

- Explain asynchronous communication behavior.
- Differentiate hardware handshaking lines.
- Illustrate lower-level data register pathways.
- Understand context saves during ISR.

Target Practical Skills

Upon completion of Unit 4, the engineering student will possess the skill to write clean logic flowcharts for polling routines, configure simple registers, and analyze cycle-stealing delays in DMA setups.

TOPIC 1: I/O ORGANIZATION FOUNDATIONS

Topic Educational Objective

"To introduce students to the structural mechanisms that coordinate and control communication paths between central processors and physical peripheral hardware systems."

Target Learning Outcome

"Students will be able to explain the core function of the data path, status check, and speed coordination layers required for stable system operation."

DEFINING I/O ORGANIZATION

Asynchronous Communication Framework: The structural assembly of bus lines, handshaking logic, timing controllers, and physical interfaces that organize data transfers between primary computational registers and external systems.

Core Responsibility 1

Speed Buffering: Resolving the speed barrier between the fast CPU clock and mechanical keyboards/disks.

Core Responsibility 2

Format Translation: Transforming sequential analog waveforms or serial bit streams into the parallel formats required by internal system buses.

WHY PERIPHERALS NEED AN INTERFACE

Logical Obstacles

A direct connection from CPU internal buses to a device is impossible due to:

- Different operational voltage standards.
- Varying physical serial interfaces.
- CPU bus clocks operating in GHz while devices operate in Hz.

Real-Life Analogy: Think of the CPU as a highly busy CEO and the devices as visitors. A reception desk (I/O Interface) handles visitors to protect the CEO's schedule.

Key Coordination Factors

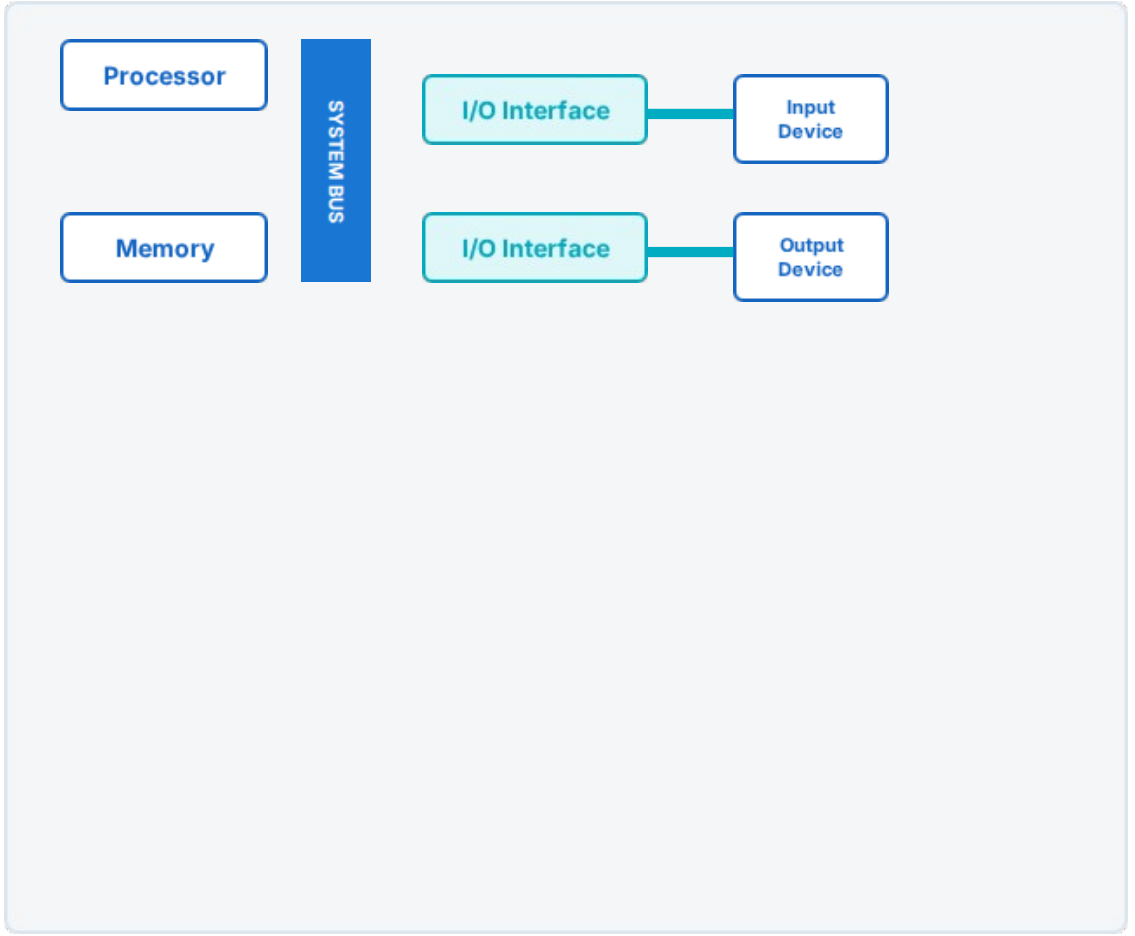
Mismatch Point	Direct Connection Consequence
Speed	CPU remains frozen waiting for input data.
Data Format	System bus registers experience line faults.
Voltages	Overvoltage risks damage to modern microchips.

BASIC I/O SYSTEM ARCHITECTURE

System Block Path

This diagram represents how the **System Bus** isolates physical peripherals from RAM and CPU registers using the **I/O Interface layer**.

- **Blue Lines:** Direct system memory buses.
- **Cyan Lines:** Buffered asynchronous data paths.



INPUT & OUTPUT DATA FLOWS

Input Path (e.g., Key Press)

Data travels inward from the physical hardware switches to system memory:

Key Switch → Interface → Bus → CPU

Output Path (e.g., Pixel Write)

Processed numerical information flows outward toward display modules:

CPU/RAM → Bus → Interface → Display

UNIT 4 DAILY QUIZ - SESSION 1

Q1 Define the speed mismatch problem in computer architecture.

Q2 Name the functional layer that isolates CPU buses from peripherals.

Q3 What logical failure occurs if dynamic voltage levels are uncoordinated?

Q4 Illustrate how parallel lines translate to a serial device interface stream.

Academic Link: https://ocw.upj.ac.id/files/Textbook-TIF203-Ebook-William-Stallings-Designing-for-Performance-8th-Edition-Prentice-Hall-2011.pdf

TOPIC 2: INPUT DEVICES & CONTROL

Topic Educational Objective

"To explain the logical translation models used by common Input Devices to convert physical real-world spatial actions into digital binary codes."

Target Learning Outcome

"Students will gain capability to explain code generation sequences in Keyboard matrices and Optical Mouse DSP chips."

INTRODUCTION TO INPUT DEVICES

Classification

Input devices translate physical variables (pressure, light, displacement, sound) into uniform binary words.

- **Alphanumeric:** Keyboard matrix boards.
- **Pointing Devices:** Optical Mouse spatial vectors.
- **Digitizers:** Scanners and Analog-to-Digital voice systems.

Functional Conversion Steps

Step	Action Type	Data Output
1	Physical Interaction	Analog Change
2	Transduction	Voltage Signal
3	ADC/Encoding	Binary Scan Word

KEYBOARD ENCODING MATRIX

Matrix Grid Scanning

Keyboards do not have separate lines for each key. They use a matrix grid of rows and columns mapped by a local processor (e.g., Intel 8048).

> **Scan Code:** A unique numerical byte generated when a key is pressed (Make Code) or released (Break Code).

KEYBOARD EXECUTION STEP-BY-STEP

How the keypress "A" is handled by computer systems:

1. Press Circuit Closes → 2. Matrix Scan Mapped → 3. Scan Code Generates → 4. Interrupt Triggered → 5. OS translates ASCII

Class Activity Question

"*What occurs inside the keyboard controller when key bouncing causes multiple continuous circuit contacts within microseconds?*"

Discuss with your class how hardware debounce delay loops solve this issue.

Key Takeaways

- Scan codes prevent character lock issues.
- Asynchronous hardware interrupts ensure characters are not dropped.

THE OPTICAL MOUSE SYSTEM

Displacement Translation

The optical mouse relies on an optoelectronic sensor taking thousands of surface photos per second to analyze direction vectors.

> **Digital Signal Processor (DSP):** Processes optical flow frames to detect differences in coordinates (\(\Delta X, \Delta Y\)) to feed computer cursor registers.

INPUT DEVICES ANALYSIS & QUIZ

Device	Signal Type	Logic Code Format	Controller Chip Target
Keyboard	Serial Electrical State	Scan Code Byte	Local Matrix Processor
Optical Mouse	Optical Flow Frame State	Coordinate Bytes (\(\Delta X, \Delta Y\))	DSP Optical Processor
Microphone	Analog Audio Signal	Digital Pulse PCM Code	ADC Sound Controller

Q1 Compare keyboard make codes and break codes.

Q2 State the function of DSP chips in optical mice.

TOPIC 3: OUTPUT DEVICES & VISUALS

Topic Educational Objective

"To analyze technical translation models required by Output Devices to project processed memory variables onto displays or physical paper sheets."

Target Learning Outcome

"Students will be able to construct logical block layouts for Display frame buffers and compare impact versus non-impact printing mechanisms."

INTRODUCTION TO OUTPUT PERIPHERALS

Output Classification

Output devices receive digital vectors and project them into formats perceivable by human senses.

- **Soft Copy:** Temporary displays (LED Monitors, CRT, Projectors).
- **Hard Copy:** Physical, permanent output formats (Impact/Non-Impact Printers).
- **Aural Output:** Audio DAC speaker units.

The Computational Interface

Every output device relies on an **output interface register** to translate CPU data words into electrical signals (pixels, drum charge levels, or magnetic voice coil movements).

MONITOR TECHNOLOGY BASICS

Pixel Matrices & Frames

Monitors map numerical data stored in a dedicated portion of memory, called the **Frame Buffer**, onto pixel matrices.

- **Resolution:** Physical pixel count (Width x Height).
- **Refresh Rate:** Display rewrite frequency (Hz).

> **Frame Buffer:** Dedicated high-speed Video RAM (VRAM) that stores color codes for screen presentation.

DISPLAY PROCESSING PIPELINE

The data processing pipeline when displaying a PowerPoint slide:

1. App calculates layouts → 2. GPU renders bitmap → **3. Writes to VRAM Buffer** → 4. HDMI sends RGB stream → 5. Pixels illuminate on display

Projector Readability Target

To ensure high visibility on classroom projectors, keep slide titles at 32px-40px, body font sizes above 16px, and implement high contrast color pairings (like dark blue text on pure white backgrounds).

Classroom Discussion

"*Why does screen tearing occur if the GPU writes to the Frame Buffer faster than the monitor's refresh rate clock?*" Explain the concept of V-Sync.

PRINTERS: HARD COPY SYSTEMS

Classification

Printers are categorized into impact or non-impact categories based on their physical mechanics.

Mechanics Definition

Impact: Physically strikes a ribbon against paper (e.g., Dot Matrix).

Non-Impact: Uses electrostatic toner, thermal transfer, or liquid ink droplets without physical contact (e.g., Laser, Inkjet).

PRINTER TECHNOLOGY COMPARISON

Printer Type	Operational Mechanics	Speed Metrics	Noise Levels	Best Implementation Case
Dot Matrix	Impact pins striking carbon ribbon	Slow	High	Carbon copy bills/receipts
Inkjet	Piezocrystal spraying liquid ink dots	Medium	Low	High detail photo graphics
Laser	Electrostatic drum fusing dry toner powder	Extremely Fast	Low	Corporate text documents

Dot Matrix Core Advantage

The dot matrix printer remains relevant in industrial supply chains due to its ability to generate carbon copies of invoices using impact printing pressure.

Laser Electrostatic Principle

Laser printers leverage positive/negative charge maps on a photosensitive drum, attracting toner particles that are fused onto paper using heated rollers.

OUTPUT ASSESSMENT & ASSIGNMENTS

Daily Quiz

1. What is the role of a display frame buffer?
2. Explain why impact printers remain in industrial billing use.
3. Differentiate between LED and OLED panel mechanics.

Weekly Assignment

Draft a 3-page engineering report comparing:

The performance overhead on the CPU during high-resolution display updates when using standard System RAM versus dedicated graphics card memory (VRAM).

TOPIC 4: THE I/O INTERFACE

Topic Educational Objective

"To examine the registers, control logic, and address decoders that make up the internal architecture of an I/O Interface module."

Target Learning Outcome

"Students will gain capability to map CPU bus lines to the Data, Status, and Control registers of an I/O interface."

DEFINING THE I/O INTERFACE

I/O Interface Module: A hardware block with address decoders and registers that sits between the system bus and a device, acting as a translator for status, command, and data signals.

Key Responsibilities

- **Address Decoding:** Recognizing if the CPU is referencing this interface's registers.
- **Command Decoding:** Interpreting write, read, or status test operations.
- **Timing & Control:** Implementing hardware handshaking lines.

The Register Concept

The interface exposes specific register slots (Data, Status, Control) directly to the CPU's address bus, allowing the processor to manage devices using standard read/write instructions.

WHY THE SYSTEM BUS NEEDS INTERFACES

Signal Translation Hurdles

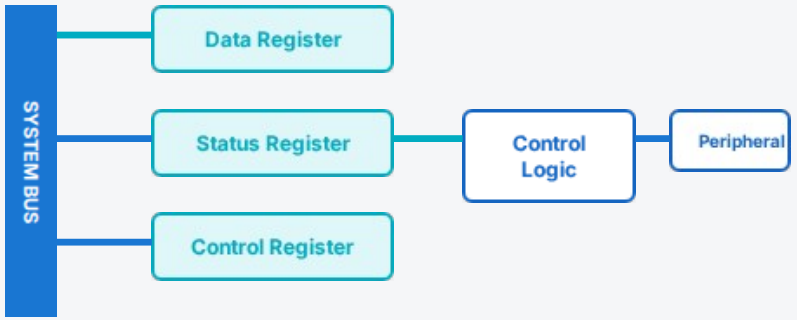
Direct processor access to devices fails due to:

- CPU bus structures operating with 64-bit parallel lines while devices may require serial bitstreams.
- Incompatible device electrical voltage standards.
- Devices operating at slower speeds than the CPU system clock.

The Speed Buffer Analogy

Express train vs country path: The system bus is a high-speed express train, and the peripheral device is a narrow country road. The I/O interface acts as a station loading platform, buffering passengers (data) so the train doesn't have to stop on the tracks.

INTERNAL INTERFACE LAYOUT



Register Explanations

- **Data Register:** Temporarily holds input or output data words.
- **Status Register:** Holds status flags (Ready, Busy, Error) read by the CPU.
- **Control Register:** Stores operational commands (Write, Start, Seek) sent by the CPU.
- **Control Logic:** Decodes address lines and manages physical lines linked to the device.

INTERFACE EXECUTION SEQUENCE

The register sequence during a CPU-to-printer data write:

1. CPU reads Status Register → 2. Status is "Ready" → 3. CPU writes to Data Register → 4. CPU writes "Print" to Control → 5. Interface prints data byte

Asynchronous Control

The status register serves as the primary feedback loop, enabling the CPU to monitor slow mechanical processes through simple memory reads.

Classroom Check

"*What occurs if the CPU writes a second data word into the Data Register before the device clears the Busy bit in the Status Register?*" Discuss data overwrite errors.

INTERFACE ASSESSMENT & RESOURCE

Q1 State the role of the Status Register inside the I/O interface.

Q2 Explain how address decoding logic acts as a device selector.

Learn Online: https://www.youtube.com/watch?v=gy_LyA1RMJw&list=PL3R9-um41JswbjLXOmzkH7AUo66lChms0

TOPIC 5: DATA TRANSFER TECHNIQUES

Topic Educational Objective

"To compare the structural layouts and execution speeds of Programmed I/O, Interrupt-Driven I/O, and Direct Memory Access (DMA)."

Target Learning Outcome

"Students will gain capability to select the optimal data transfer method based on system design and device speed."

UNDERSTANDING DATA TRANSFER OPTIONS

To manage data transfers between devices and memory, computer architectures support three primary communication techniques:

1. Programmed I/O

The CPU is directly responsible for data transfers, continuously checking device status in a loop (Polling).

2. Interrupt-Driven

The device interface issues an interrupt request when ready, freeing up CPU cycles for other tasks.

3. DMA Module

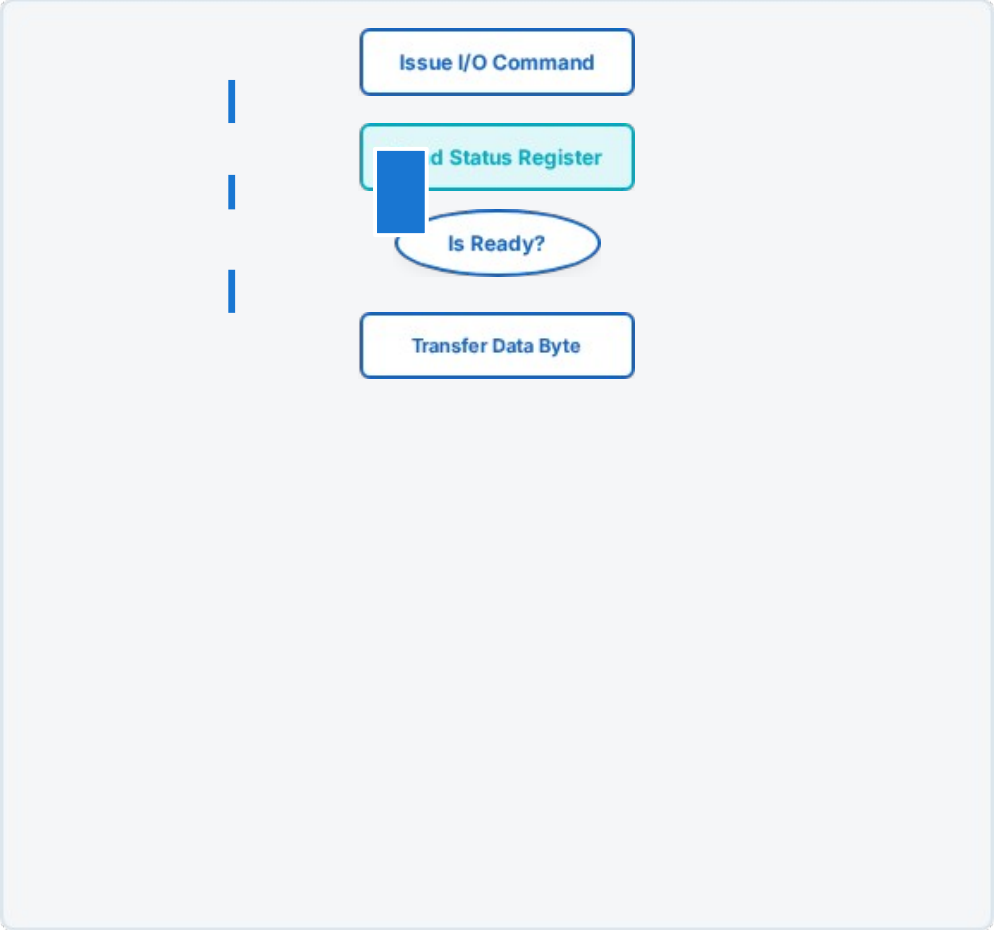
A specialized hardware controller handles data transfers directly between the device and memory, bypassing the CPU.

PROGRAMMED I/O & POLLING

Polling: A software loop in which the CPU continuously reads the interface Status Register to check if the device is ready for data transfer.

The Cost of Busy Waiting

During polling, the CPU is stuck in a loop, wasting clock cycles that could be used for other computation.



PROGRAMMED I/O WORKING CASE

Busy Waiting Cost Analysis

Consider a CPU writing to a 1200-baud printer (120 bytes per second) with an instruction execution rate of 10 million MIPS:

- The printer requires ~8.3 milliseconds per character write.
- The CPU wastes up to **83,000 instructions** per polling loop per character.

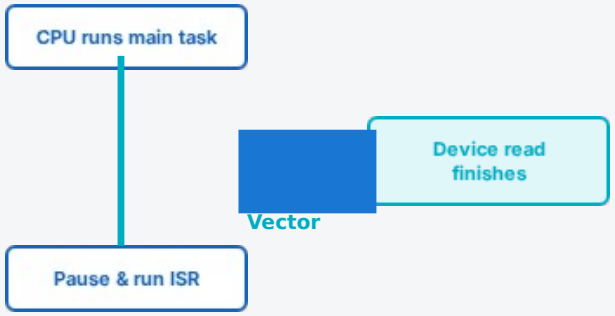
Real-Life Analogy: Polling is like standing by your front door and opening it every 5 seconds to check if a delivery has arrived.

INTERRUPT-DRIVEN I/O

Interrupt-Driven Transfer: An asynchronous hardware technique where the CPU issues an I/O command and continues other work. The device interface then interrupts the CPU only when it is ready to transfer data.

Key System Gains

This approach eliminates busy waiting. The CPU only pauses its current task when the device is ready, optimizing clock cycles.



INTERRUPT-DRIVEN WORKINGS

System Handling Steps

The sequence when a keyboard key is pressed:

1. The user presses a key, generating a scan code in the interface buffer.
2. The interface asserts the physical **Interrupt Request (IRQ)** line.
3. The CPU completes its current instruction and recognizes the interrupt.
4. The CPU saves its execution context, runs the Keyboard ISR to read the scan code, and restores its context.

The Doorbell Analogy

Interrupt-driven I/O is like sitting at your desk working on an assignment. When a package is delivered, the visitor rings the **doorbell**. You pause your work, collect the package, and immediately return to your assignment.

TOPIC 8: DIRECT MEMORY ACCESS

Topic Educational Objective

"To explain the architecture, register configurations, and bus-grant handshakes of the Direct Memory Access (DMA) controller."

Target Learning Outcome

"Students will be able to trace data transfers through a DMA controller during high-speed block transfers, bypassing the CPU."

UNDERSTANDING DIRECT MEMORY ACCESS

Direct Memory Access (DMA): A hardware technique where a dedicated controller chip (DMAC) takes control of the system bus to transfer blocks of data directly between a peripheral device and memory, bypassing the CPU.

Why DMA is Required

Interrupt-driven transfers are inefficient for high-speed block transfers (e.g., reading from an SSD or network card) because the CPU must still process every single byte via an ISR, wasting cycles.

DMAC Core registers

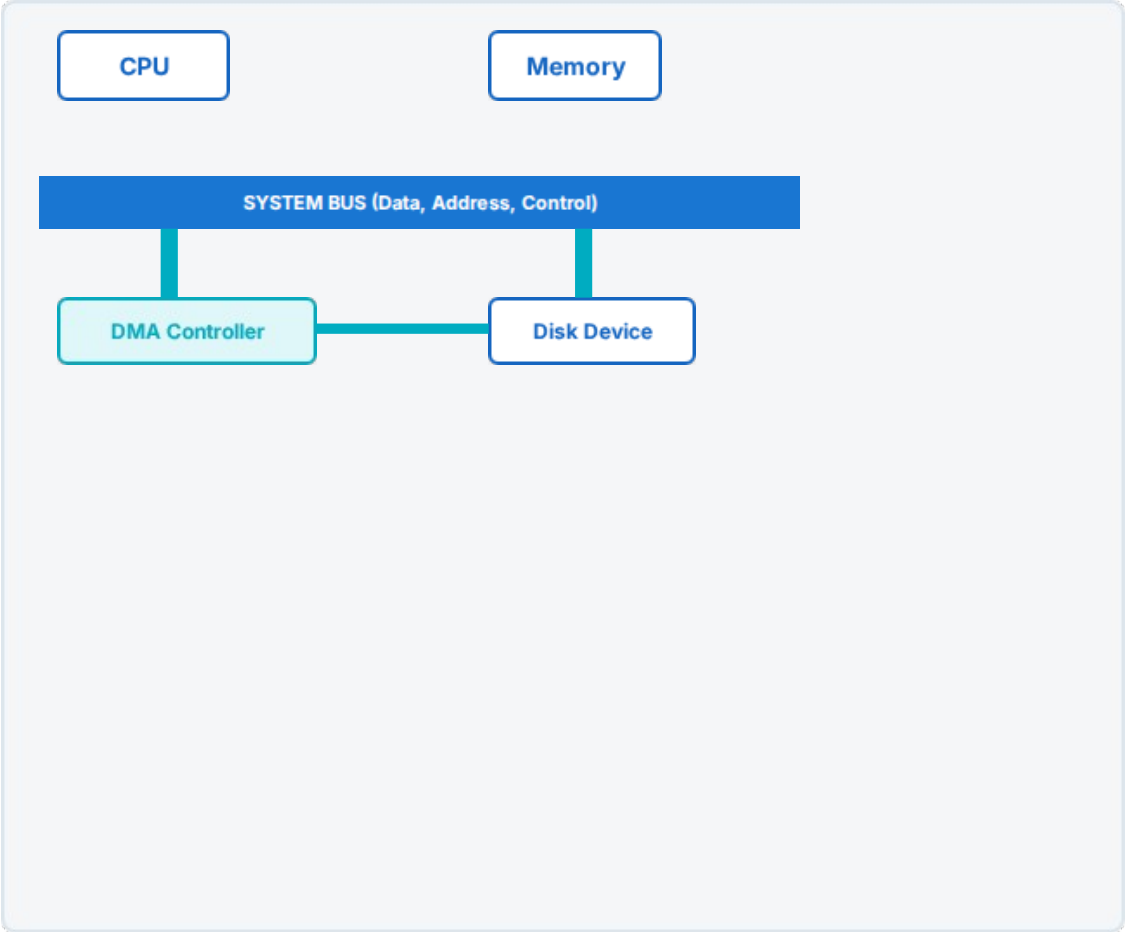
- **Address Register:** Holds the starting memory address.
- **Count Register:** Stores the block size (number of words) to transfer.
- **Control Register:** Manages transfer options (Read, Write, Burst).

DMA CONTROLLER SYSTEM CONNECTION

Direct Memory Paths

This diagram represents how the **DMA Controller (DMAC)** takes control of the system bus to transfer data directly between memory and devices.

- **Blue Lines:** Initialization and Bus Grant lines.
- **Cyan Highlighted Path:** High-speed direct data transfer.



STEP-BY-STEP DMA TRANSFER

The complete register and bus sequence for a DMA transfer:

1. CPU configures DMAC registers → 2. Device asserts DMA Request → **3. DMAC requests Bus Control** → 4. CPU grants Bus control → 5. Data transfer directly to RAM

Bus Grant Handshake

The CPU uses the **Hold Request (HOLD)** and **Hold Acknowledge (HLDA)** signals to hand bus control to the DMA controller, tri-stating its own bus output lines.

Completion Notification

When the word count register reaches zero, the DMA controller releases bus control and sends an interrupt request to the CPU to flag transfer completion.

DMA BLOCK TRANSFER EXAMPLE

SSD-to-RAM Block Transfer

Consider loading a 4096-byte page from an SSD into system RAM.

- **Without DMA:** The CPU runs a loop 4096 times to read, buffer, and write each byte, blocking other tasks.
- **With DMA:** The CPU configures the start address and byte count, and the DMA controller handles the block transfer directly, leaving the CPU free for other computation.

The Delivery Supervisor Analogy

Think of the CPU as a busy Store Manager and the DMA controller as a Delivery Supervisor. Instead of the manager carrying 100 boxes from the delivery truck to the warehouse one by one, the manager tells the supervisor to handle the move and report back once the job is complete.

DATA TRANSFER METHODS COMPARED

A comparison of the three primary data transfer techniques:

Feature	Programmed I/O	Interrupt-Driven I/O	DMA Transfer
CPU Involvement	High (Continuous loop)	Medium (Interrupt processing)	**Low (Configuration only)**
Hardware Complexity	Minimal (No extra hardware)	Low (Standard interrupt line)	High (Dedicated DMA chip)
Polling Overhead	**Yes (Busy waiting)**	No	No
Data Path	Device → CPU → RAM	Device → CPU → RAM	**Device ↔ RAM (Direct)**
Target Applications	Simple, low-cost microcontrollers	Low-speed devices (Mouse, Keyboard)	High-speed devices (SSD, GPU)

DATA TRANSFER ASSESSMENT

Daily Quiz

1. Why is programmed I/O inefficient for high-speed devices?
2. State the function of HOLD and HLDA signals.
3. Explain the term 'cycle stealing' in DMA transfers.

Weekly Assignment

Complete the following assignment:

Draw a detailed flowchart comparing the execution path of an interrupt-driven transfer versus a DMA block transfer, showing bus ownership shifts.

Topic DMA: https://youtu.be/qiaEGFh4D_I?si=uhCgYvZmSk6SUbSt

TOPIC 9: INTERRUPT SYSTEMS

Topic Educational Objective

"To explain the logical classifications, priority resolution schemes, and execution vector routines of system interrupts."

Target Learning Outcome

"Students will be able to trace context switches and priority level mapping rules when multiple interrupt requests overlap."

DEFINING SYSTEM INTERRUPTS

System Interrupt: An asynchronous signal sent to the CPU by hardware or software, flagging an event that requires immediate processing by interrupting the main program execution flow.

Core Concepts

- **ISR:** The Interrupt Service Routine (or handler) that contains the instructions to process the interrupt.
- **Context Saving:** The CPU automatically saves its execution state (Program Counter and flags) to the system Stack before jumping to the ISR.

The Alarm Analogy

Think of an interrupt as a safety alarm in a factory. The main system runs standard assembly line tasks, but if a safety alarm (interrupt) goes off, the system pauses immediately, addresses the alarm issue, and then resumes assembly operations.

INTERRUPT CLASSIFICATIONS

Hardware Interrupts

Triggered by physical signals from external devices connected to the processor's IRQ pins (e.g., keyboard presses, timer updates).

Software Interrupts

Triggered directly by execution of software instructions (e.g., assembly INT calls for OS services).

Maskable Interrupts

Interrupts that can be temporarily disabled or ignored by the CPU when executing time-critical code blocks.

Non-Maskable Interrupts (NMI)

High-priority interrupts that cannot be disabled or ignored by software (e.g., power loss warnings, hardware failures).

INTERRUPT HANDLING & MCQS

The Context Switch Sequence

1. Processor completes the current instruction.
2. CPU pushes PC and Status Register flags to the Stack.
3. Vector address is decoded to locate the target ISR.
4. CPU branches to and executes the ISR handler.
5. The ISR ends with an RTI (Return from Interrupt) instruction.
6. Saved state is popped from the Stack, resuming the main program.

Unit 4 MCQ Practice

1. Which register buffers input data before bus transfer?
A) Status Register B) Control Register C) Data Register

2. Which scheme tri-states CPU lines for direct transfers?
A) Polling B) DMA Controller C) Programmed I/O

3. What type of interrupt request cannot be disabled?
A) Maskable B) Software C) Non-Maskable (NMI)

UNIT 4: MULTIPLE CHOICE QUESTIONS

QUESTION 1

Which register buffers input data before it is transferred to the system bus?

A) Status Register

B) Control Register

C) Data Register

QUESTION 2

Which I/O data transfer scheme tri-states the CPU lines to perform direct memory transfers?

A) Polling

B) DMA Controller

C) Programmed I/O

QUESTION 3

What type of interrupt request cannot be disabled or ignored by the processor?

A) Maskable Interrupt

B) Software Interrupt

C) Non-Maskable Interrupt (NMI)

QUESTION 4

Which instruction is executed at the end of an Interrupt Service Routine (ISR) to restore CPU state?

A) RET (Return)

B) RTI (Return from Interrupt)

C) IRET (Interrupt Return)

EXAM PREP & UNIT RECAP

Key University Exam Questions

- Draw and label the block diagram of a basic I/O Interface.
- Compare Programmed I/O, Interrupt-Driven I/O, and DMA.
- Trace the context-saving sequence during interrupt handling.
- Explain the differences between burst and cycle-stealing DMA modes.

Thank You!

Newton School of AI & Computer

Presented by: **Deepanshu Saini**

> **Unit Recap:**
> Remember: I/O Interfaces are speed and format translators. Polling wastes CPU cycles. Interrupts assert asynchronous event requests, and DMA controllers handle direct high-speed block data transfers to RAM, bypassing the CPU.

Thank You!

Questions & Discussion